

LLM-Enhanced Predictive Maintenance on NASA C-MAPSS

Full Project Report — FD001 baseline + FD002/FD004 extension

Part 1 — FD001 Baseline

1. Model performance vs. published benchmarks

Trained a 2-layer LSTM (hidden size 64, dropout 0.3) on NASA C-MAPSS **FD001** (100 train engines, single operating condition, HPC degradation fault mode), using a 30-cycle sliding window, piecewise-linear RUL target capped at 125 cycles, and 14 non-constant sensors (7 sensors — 1, 5, 6, 10, 16, 18, 19 — are physically near-constant at a single operating condition and were dropped).

Model	Dataset	RMSE	NASA Score	Source
This project (LSTM)	FD001	13.58	335.0	—
LSTM	FD001	16.78	382.4	Pinto et al., 2023
CNN-LSTM	FD001	16.78	382.4	penikmatrump, 2024
Transformer Encoder	FD001	15.08	338	Gsebs, 2023

(Benchmark rows sourced live by the Literature Agent's web search — see `models/results/literature_knowledge_base.json` for full citations.)

Our model's RMSE (13.58) beats every benchmark the Literature Agent found, and its NASA score (335.0) is close to the best of them. This is a genuine, reproducible result — rerun with `python -m ml.train_lstm`.

2. Explainability + literature cross-check pipeline

For all 100 test engines, SHAP (`GradientExplainer`) attributes the RUL prediction to specific sensors (`ml/explain_shap.py`). The 15 highest-risk engines (lowest predicted RUL, see `ml/risk_scoring.py`) get a full multi-agent analysis:

- Literature Agent** (OpenAI `gpt-4o-mini` + web search) — researches real published C-MAPSS papers once, builds a knowledge base of which sensors are reported as predictive and what degradation patterns look like. Grounded with the canonical NASA sensor-index table so it can't mismap a paper's physical sensor names (T30, P30, Nc, ...) to the wrong `sensor_N`.
- Synthesis Agent** (Cohere `command-a-03-2025`) — compares each engine's top SHAP sensors against the knowledge base and labels the case Confirmed / Contradicted / Novel.
- Hypothesis Agent** (OpenAI `gpt-4o-mini`) — only runs on Contradicted/Novel cases, proposes a plausible explanation + next checks.
- Critique Agent** (Cohere `command-a-03-2025`, a different model/lab than the rest of the pipeline) — independently reviews a 15-engine sample, actively looking for a simpler model-setup explanation (SHAP

background artifact, low sensor variance, extrapolation) rather than agreeing by default.

Result on the 15 highest-risk engines

Verdict	Count
Confirmed	2
Contradicted	0
Novel	13
Critiqued (independent 2nd opinion)	15

This is the project's genuine, reportable finding. The dominant outcome is Novel, not because the pipeline defaults to it, but because the knowledge base — built from real papers the Literature Agent could actually find and verify — only documents 5 of the ~14 informative C-MAPSS sensors in depth. Where the model's top driver *did* overlap with a documented sensor (`sensor_7/P30`, `sensor_9/Nc`, `sensor_12/phi`, `sensor_14/NRc`), the pipeline correctly confirmed agreement (0 contradictions) and cited its source. Where the dominant SHAP sensor (most often `sensor_11/Ps30`) fell outside that coverage, it correctly declined to force a verdict rather than fabricate one — see `models/results/agent_analysis/engine_34.json` for a representative example, including the Critique Agent independently pushing back with an alternative "SHAP artifact / distribution extrapolation" explanation.

Known limitation

Literature Agent's web search cannot read figures — early runs mapped a paper's physical sensor names to the wrong `sensor_N` index (citing sensors that are known-constant in FD001). Fixed by embedding the canonical Saxena & Goebel (2008) sensor table directly in its system prompt. Worth noting in any write-up as a concrete example of LLM-literature-grounding failure mode and its fix.

3. How to reproduce

```
## 1. Train + evaluate the LSTM
python -m ml.train_lstm

## 2. SHAP explanations for all 100 test engines
python -m ml.explain_shap

## 3. Rank engines, flag top 15 for deep analysis
python -m ml.risk_scoring

## 4. One-time literature research (requires OPENAI_API_KEY)
python -c "from app.agents.literature_agent import build_knowledge_base; build_knowledge_base()"

## 5. Run the agent pipeline on the flagged engines (requires OPENAI_API_KEY, COHERE_API_KEY)
cd backend && python -m app.agents.graph
]
```

Dashboard: `backend` (FastAPI, port 8000) + `frontend` (Next.js, port 3000). See `.claude/launch.json`.

Part 2 — FD002 & FD004 Extension

Evaluation criteria and model settings were written down before any result existed: docs/FD002_FD004_preregistration.md (timestamped; later additions are dated addenda, each made before the stage it affects).

Headline numbers

Test RMSE / PHM08 score on every test engine (last-cycle prediction).

Subset	Test engines			RMSE (truth capped at 125)	Score (capped)	RMSE, engines with true RUL <= 125
FD001	100	14.90	371	13.58	335	13.63
FD002	259	28.38	17,873	14.99	1,101	13.84
FD004	248	27.28	5,221	15.08	1,096	15.76

Primary numbers are against the raw provided RUL, as the spec requires. FD001's previously reported 13.58 was against truth capped at 125; its raw equivalent is 14.90.

Why the raw and capped numbers differ so much on FD002/FD004: the model is trained on a piecewise RUL target capped at 125 (the FD001 setting, kept unchanged), so it cannot predict above ~125. 22% of FD002 test engines (57/259, max 194) and 27% of FD004 (67/248) have a true RUL above 125, against 11% for FD001. Every one of those is an unavoidable early-prediction error. Restricting to engines where the cap cannot matter isolates the model's actual skill: FD001 13.63, FD002 13.84, **FD004 15.76** — FD004 is genuinely harder, FD002 about as hard as FD001 once operating conditions are normalized out.

1. Sensor selection — FD001's choice does not transfer

FD001 dropped sensors 1, 5, 6, 10, 16, 18, 19 as near-constant. Re-checked on FD002/FD004 training data (models/results/FD00x/sensor_audit.json):

- **Globally, all seven look highly variable** (sensor_1 std 26.4) — but that is entirely the six flight regimes shifting them. A global variance check would wrongly keep all seven.
- **Inside a regime, sensors 1, 5, 18, 19 take exactly one value** in both subsets: pure regime indicators, no degradation information. Dropped.
- **Sensors 6, 10, 16 are NOT constant inside regimes, and carry signal.** They are quasi-binary (one value 77–97% of the time), but the deviations track degradation: in-regime Spearman rho vs RUL, FD002 / FD004: sensor_6 -0.36 / -0.21, sensor_10 +0.09 / -0.22, sensor_16 -0.62 / -0.31. **Kept** — three sensors FD001 excluded.

Rule applied (same principle as FD001, now within regime): drop a sensor only if it is constant inside every regime. Inputs: 17 sensors + 6 regime flags.

This decision turned out to matter a lot: in FD004, **sensor_6 (P15, bypass-duct pressure) is the top SHAP driver for 98 of 248 engines** — more than any other sensor. Bypass flow is driven by the fan, and FD004's second fault mode is fan degradation. Copying FD001's sensor list would have hidden the fan fault from the model. In FD002 (HPC fault only), sensor_11 (Ps30, HPC static pressure) is the top driver for 238 of 259 engines, matching FD001.

The second fault mode, visible in the raw data. Sensors 7, 12, 20, 21 correlate with RUL at $|\rho| \sim 0.45\text{--}0.50$ in FD002 but only 0.06–0.12 in FD004: HPC and fan degradation move them in opposite directions, so pooled across FD004 engines their signal largely cancels.

2. Normalization — per operating condition

k-means ($k = 6$) on the three operational settings, fit on training rows only. Clusters are unambiguous: every train and test row lies within 0.0053 of its centroid, and all six regimes are well populated (8,002–15,395 train rows). Each kept sensor is z-scored within its regime using training statistics; the FD001 global min-max scaling was not reused.

One adjustment, made after a smoke test and before training (addendum in the pre-registration): the quasi-binary sensors reach $|z| \sim 50$ because their in-regime std is tiny, while continuous sensors peak at $|z| \sim 8.6$. z-scores are clipped to ± 10 , which caps the flips without touching any continuous sensor.

3. Model — trained from scratch, not transfer-learned

Transfer from the FD001 model is not methodologically sound here:

1. **Different input space.** The FD001 model takes 17 globally min-max-scaled columns; FD002/FD004 inputs are 23 columns (three sensors FD001 never saw, six regime flags) on a different scale. Reusing its weights would map inputs to meanings they don't have.
2. **Transfer solves data scarcity, and there isn't any.** The targets (221 / 212 training engines after validation hold-out) are larger than the source (100 engines).
3. **FD004's fan fault never occurs in FD001.** A source model can't carry knowledge of a failure mode it has never seen, and fine-tuning from it risks anchoring on HPC-only degradation patterns.
4. **Comparability.** The published per-subset benchmarks train on that subset alone; a transfer model would not be a like-for-like comparison.

Same architecture as FD001 (2-layer LSTM, hidden 64, dropout 0.3, window 30, cap 125, Adam 1e-3, batch 256, seed 42). Three pre-declared changes: engine-level validation split (FD001 split windows randomly, so an engine could appear in both train and validation — this only affected FD001's early stopping, never its test set); target scaled by 125 (FD001 training sat on a flat plateau for ~20 epochs before learning); patience 15 / max 100 epochs.

	Train / val / test engines	Best epoch	Val RMSE at best	Stopped at
FD002	221 / 39 / 259	6	16.27	21
FD004	212 / 37 / 248	13	14.65	28

Both overfit after the best epoch (train RMSE fell to ~10 while validation rose); the best-validation checkpoint is what was evaluated.

4. Published benchmarks — audited, subset-specific

The Literature Agent was run separately for each subset. **Its output failed audit and is not used unverified.** Every row was checked against its source:

- FD002 "Transformer 15.08" — fabricated; the cited page reports 17.21 (Transformer) and 18.73 (LSTM) for FD002. 15.08 matches the FD001 figure found earlier.
- FD002 "Stacking ensemble 8.613" (arXiv 2608.27940) — that is the paper's **FD003** result; it reports no FD002 number.
- FD002 rows from aitrendblend.com — genuine FD002 numbers, but the agent invented the method names (its "LSTM 13.22" is an attention LSTM, DA-LSTM; its "CNN-LSTM 14.3" is a graph network, ASTHGNN).
- FD004 "BACE-RUL / Cox-PH / SVM / LSTM" rows — an unexplained "custom score" in the hundreds of thousands, not the PHM08 score. Excluded.
- FD002 knowledge base cited sensors 5 and 19 — constant inside every FD002 regime, so no finding can apply to them. Removed by a pre-declared rule.

Raw agent output is kept as `literature_knowledge_base.raw.json`.

Primary benchmark source — one paper, one comparison table, consistent protocol: Ragab et al., *Attention Sequence to Sequence Model for Machine Remaining Useful Life Prediction*, arXiv:2007.09868, Table III. Its FD001 LSTM row (16.14 / 338) matches the FD001 benchmark verified earlier.

Method (Ragab et al. Table III)	FD002 RMSE / Score	FD004 RMSE / Score
D-LSTM (Zheng et al. 2017) — plain LSTM	24.49 / 4,450	28.17 / 5,550
BLSTM	25.11 / 4,793	26.61 / 4,971
1D CNN (Li et al. 2018)	22.36 / 10,412	23.31 / 12,466
BiLSTM-ED	22.07 / 3,099	23.49 / 3,202
BLCNN	19.09 / 1,558	20.97 / 3,859
HDNN	15.24 / 1,282	18.16 / 1,527
ATS2S (the paper's method)	14.65 / 876	16.66 / 1,074
This project (raw truth)	28.38 / 17,873	27.28 / 5,221
This project (truth capped at 125)	14.99 / 1,101	15.08 / 1,096

Secondary, recent state of the art (secondary sources, not independently verified against the original papers): DCPGCN, Zheng et al. 2026, *Advanced Engineering Informatics* — FD002 12.06 / 557.93, FD004 13.64; RGPD (2025) — FD004 12.38 / 787.39.

Honest reading. Which truth convention the benchmark papers use for the *test* labels is not stated in Ragab et al. (they cap *training* labels at 130 for FD002/FD004). That decides the comparison:

- If the benchmarks are scored against capped truth — the common practice in deep-learning C-MAPSS papers, and consistent with scores in the hundreds to low thousands — this plain LSTM beats the plain-LSTM baseline by a wide margin (FD002 14.99 vs 24.49; FD004 15.08 vs 28.17) and lands among the hybrid models, with the gain coming from per-regime normalization and the corrected sensor set rather than the network. It does not reach 2024–26 state of the art.
- Against raw truth, it is worse than every benchmark in the table. That is the pre-registered primary number and it is reported as such.

5. Agent verdicts

Synthesis -> Hypothesis ran on every test engine, as the spec required: all 259 FD002 and all 248 FD004 engines (507 total), not a sample.

Subset	n	Confirmed	Contradicted	Novel	Novel rate
FD001 (top-15, original)	15	2	0	13	86.7%
FD002 (all 259)	259	0	60	199	76.8%
FD004 (all 248)	248	55	24	169	68.1%

Pre-declared criterion ("same pattern" = Novel rate within FD001's 86.7% $\pm$ 15pp, i.e. 71.7–100%):

- **FD002: 76.8% — inside the range. Criterion says the pattern holds.** But that number hides something the criterion doesn't check for: **zero Confirmed verdicts across 259 engines.** Cause, not coincidence: sensor_11 is FD002's #1 SHAP driver for 238/259 engines (Section 1), and FD002's literature knowledge base contains no finding for sensor_11 at all — only three low-importance sensors (10, 15, 16) survived the validity filter. With the dominant driver absent from the knowledge base, most engines can only land on Contradicted or Novel; Confirmed was structurally unlikely regardless of what the model actually learned. The Novel-rate criterion passing is compatible with a **worse-covered** knowledge base, not necessarily a better-matching model — a real limitation of this criterion the project owner should know about.
- **FD004: 68.1% — outside the range (needed >=71.7%). This is a genuinely different result, as the spec requires it be called.** Two contributing, verifiable causes: (a) FD004's knowledge base is more useful — sensor_11 IS covered there ("high" importance), and sensor_11 is FD004's #2 SHAP driver (70/248 engines), which is why FD004 gets real Confirmed verdicts (55, 22%) where FD002 gets none; (b) FD004's #1 driver, sensor_6, is covered by neither knowledge base, but its physical link to the fan-degradation fault (Section 1) makes "Novel" here a substantively correct read of a fault mode the literature search didn't surface — a data point in favor of the pipeline reasoning correctly, even though it moves the metric outside the band.

Secondary, like-for-like number (top-15 highest-risk engines only, matching FD001's original scope): FD002 0 Confirmed / 8 Contradicted / 7 Novel (46.7% Novel); FD004 4 Confirmed / 0 Contradicted / 11 Novel (73.3% Novel, inside the band). The full-dataset and top-15 FD002 Novel rates disagree by 30 points — the top-15 (all imminent failures) skew more Contradicted, the full set skews more Novel; this is a real effect of engine selection, not noise.

Critique. Cohere's trial key (20 calls/min, ~1,000/month) could not cover Critique for all 507 engines in reasonable time; the project owner chose to run it on the top-15-highest-risk engines per subset instead (30 engines), matching FD001's original scope. Result: **the Critique Agent disagreed with all 30 of 30 checked verdicts**, in both this sample and an earlier partial full-dataset run (50 engines, stopped early) that showed the same 0% agreement. Reading the actual text (not just the agree/disagree flag): the critiques are genuine and case-specific — they name the actual driving sensors from each engine's SHAP result — but every one converges on the same counter-explanation: the driving sensors' importance may be a SHAP background-sampling artifact or reflect low in-regime variance/noise rather than a real signal. That is a legitimate methodological concern (Section 2's z-clipping exists because exactly this pattern shows up in the raw data), and it is not disprovable with what this pipeline currently computes — nothing here measures SHAP stability across background samples or reports per-sensor in-regime variance to the Critique Agent. Read plainly: the critique step is functioning as designed (skeptical, independent, not rubber-stamping), but a 100% disagreement rate across every sample checked also means it is not currently discriminating between engines where this concern is more or less warranted — that would need SHAP-stability or variance metrics added to what the Critique Agent is given, which is future work, not something to paper over here.

6. Deviations and limitations

- Test sets are 259 and 248 engines, not 100; all were used for Synthesis and Hypothesis. Critique ran on 30 (top-15 per subset) by the project owner's choice once the Cohere trial rate limit made a same-day full run impractical — see Section 5 for exactly which engines and what it found.
- The RUL cap of 125 (kept from FD001 for consistency) is the single biggest driver of raw-truth error on these subsets. A higher cap was not tried — that would be tuning after seeing results.
- The FD002 knowledge base is thin (three valid sensor findings, all rated "low", missing the actual #1 driver sensor_11 entirely) — this is very likely why FD002 produced zero Confirmed verdicts, and it means the Novel-rate criterion passing for FD002 should not be read as "the pipeline agreed with literature about as often as on FD001." See Section 5.
- The Critique Agent disagreed with 100% of the 30 sampled verdicts, always via the same SHAP-artifact/low-variance argument. Genuine per-case reasoning, but the pipeline gives it no way to check that argument (no SHAP-stability or in-regime-variance metric is computed or passed to it), so it cannot currently distinguish cases where the concern is warranted from cases where it isn't. Not fixed here; noted as the clearest

next step.